MAT3007H: Honours Optimization

Linear Programming Solver — Implementation Manual

Linchuan Wei

School of Data Science, CUHK-Shenzhen

March 2026

Abstract

This manual details the step-by-step implementation of a production-grade **Revised Dual Simplex** solver in C++. It bridges the gap between mathematics and high-performance software through five stages of development, covering sparse linear algebra, basis maintenance, advanced pricing strategies, hypersparsity exploitation, and presolve/postsolve techniques.

Contents

1 Phase I: Initialization and the Big-M Method **3**
- 1.1 The Bounding Constraint . . . 3
- 1.2 The Phase I Execution Cycle . . . 3
- 1.3 Step-by-Step Numerical Example . . . 4

2 Sparse Linear Algebra and Factorization **5**
- 2.1 Compressed Sparse Column (CSC) Format . . . 5
- 2.2 The Foundation: LU Factorization . . . 6
- 2.3 Forward and Backward Substitution . . . 7
- 2.4 The Sparse Matrix Dilemma and Fill-In . . . 8
- 2.5 Markowitz Threshold Pivoting . . . 8
- 2.6 Delegating Heavy Lifting to Third-Party Libraries . . . 9

3 The Revised Dual Simplex Core Operations **9**
- 3.1 Computing Pivot Row and Column . . . 9

4 Handling Hypersparsity in Triangular Solves **10**
- 4.1 The `IndexedVector` Data Structure . . . 11
- 4.2 The Gilbert-Peierls Algorithm . . . 11
- 4.3 Viewing the Matrix as a Graph . . . 12
- 4.4 Depth-First Search (DFS) and Topological Sort . . . 12
- 4.5 Step-by-Step Reachability Example . . . 13

5 Phase II: Advanced Basis Maintenance (ETA-Files) 14
5.1 The Mathematical Derivation of the Update 14
5.2 The ETA-File Structure 14
5.3 Solving against an Eta Matrix 15

6 Phase III: Advanced Dual Pricing and Ratio Tests 16
6.1 Dual Steepest Edge (`CHUZR`) 16
6.2 The Goldfarb-Reid Recurrence Relation 18
6.3 The Harris Two-Pass Ratio Test (`CHUZC`) 20

7 Phase 0: Presolve and Postsolve Architecture 20
7.1 Standard Form Presolve Reductions 21
7.2 The Postsolve Stack 22

8 System Architecture and Modular Design 23
8.1 Module 1: The Data Model (`LpData`) 23
8.2 Module 2: The Factorization Engine (`LpFactor`) 23
8.3 Module 3: The Dual Simplex Engine (`DualSimplexEngine`) 23
8.4 Module 4: Presolve and Postsolve (`Presolver`) 24
8.5 Architecture Diagram 24

1 Phase I: Initialization and the Big-M Method

The **Revised Dual Simplex** method is a highly efficient computational engine, but it requires a strict precondition: the initial basis must be **dual feasible**. In standard minimization form, this means all reduced costs must be non-negative ($\bar{c}_j \geq 0$).

When an initial model is both primal infeasible (negative basic variables) and dual infeasible (negative reduced costs), the standard ratio tests fail. To bypass this, we use the **Big-M Method**—a Phase I technique that introduces a single artificial constraint to mathematically force the entire system into a dual-feasible state after just one initial pivot.

1.1 The Bounding Constraint

Definition — The Bounding Constraint

To force dual feasibility, we introduce a single artificial constraint that bounds the sum of all non-basic variables. Let $\mathcal{N}$ be the set of initial non-basic variables, and let M be an arbitrarily large positive number. We add the constraint:

$$\sum_{j \in \mathcal{N}} x_j \leq M$$

By adding a non-negative artificial slack variable $x_a \geq 0$, we convert this into standard equality form:

$$\sum_{j \in \mathcal{N}} x_j + x_a = M$$

Note — The Nature of M

In a theoretical or chalkboard setting, M is not assigned a numerical value. It is treated algebraically as a constant that approaches infinity ($\lim_{M \to \infty}$). It dominates all other constants in the system, ensuring that any term containing $+M$ is overwhelmingly positive and $-M$ is overwhelmingly negative.

1.2 The Phase I Execution Cycle

The goal of Phase I is to execute a single, targeted "forcing pivot" that overpowers the negative reduced costs.

The Forcing Pivot

1. **Identify the Need:** Examine the objective function. If any non-basic variable has a reduced cost $\bar{c}_j < 0$, the basis is dual infeasible.
2. **Add the Constraint:** Append the bounding constraint $\sum_{j \in \mathcal{N}} x_j + x_a = M$ to the tableau. The artificial variable x_a enters the initial basis.
3. **Select the Entering Variable (q):** Choose the non-basic variable x_k that has the *most negative* reduced cost ($\min \bar{c}_j$).
4. **Select the Leaving Variable (p):** Force the artificial slack variable x_a to leave the basis.

5. **Pivot:** Execute the algebraic substitution. Because x_a leaves, the entering variable absorbs the massive M value, shifting all reduced costs to positive values. Phase II can now safely begin.

1.3 Step-by-Step Numerical Example

Let us observe the mathematics of the forcing pivot in action using a dictionary representation.

Example 1.1 — Restoring Dual Feasibility with Big-M

Consider the following minimization problem in standard form:

$$\min z = -x_1 - 2x_2$$

Subject to:

$$\begin{aligned} -x_1 + x_2 + s_1 &= -1 \\ x_1 + x_2 + s_2 &= 4 \\ x_1, x_2, s_1, s_2 &\geq 0 \end{aligned}$$

Step 1 — Assess the Initial State

- **Basic variables:** $s_1 = -1$, $s_2 = 4$. (Primal infeasible because $s_1 < 0$).
- **Non-basic variables:** $x_1 = 0$, $x_2 = 0$.
- **Reduced costs:** $\bar{c}_1 = -1$, $\bar{c}_2 = -2$. (Dual infeasible because both are < 0).

Step 2 — Add the Bounding Constraint

We introduce the artificial bounding constraint $x_1 + x_2 + x_a = M$. Rewriting the system as a dictionary, we have:

$$\begin{aligned} s_1 &= -1 + x_1 - x_2 \\ s_2 &= 4 - x_1 - x_2 \\ x_a &= M - x_1 - x_2 \\ z &= -x_1 - 2x_2 \end{aligned}$$

Step 3 — Execute the Forcing Pivot

- **Entering Variable:** x_2 (it has the most negative reduced cost, -2).
- **Leaving Variable:** x_a (by the Phase I rule).

Rearrange the bounding constraint to isolate the entering variable x_2:

$$x_2 = M - x_1 - x_a$$

Step 4 — Substitute and Update

Substitute the expression for x_2 back into the objective function:

$$\begin{aligned} z &= -x_1 - 2(M - x_1 - x_a) \\ z &= -2M + x_1 + 2x_a \end{aligned}$$

Substitute x_2 into the remaining constraints to update the basic variables:

$$s_1 = -1 + x_1 - (M - x_1 - x_a) \implies s_1 = (-1 - M) + 2x_1 + x_a$$
$$s_2 = 4 - x_1 - (M - x_1 - x_a) \implies s_2 = (4 - M) + x_a$$

Step 5 — Conclusion of Phase I

Examine the updated objective function: $z = -2M + x_1 + 2x_a$. The non-basic variables are now x_1 and x_a. Their respective reduced costs are $\bar{c}_1 = 1$ and $\bar{c}_a = 2$. Because both reduced costs are ≥ 0, **the system is now strictly dual feasible.** You may immediately proceed to Phase II (the standard Revised Dual Simplex method), starting by selecting s_1 to leave the basis since it holds the primal-infeasible value of $(-1 - M)$.

Note — Production Reality: Choosing a Numerical M

While M is treated as an infinitely large algebraic concept on paper, software implementations must assign it a concrete floating-point value (typically a 64-bit `double`). This creates a delicate numerical balancing act for solver developers:

- **If M is too small:** The artificial constraint $\sum x_j \leq M$ becomes a physical boundary. If the true optimal solution requires a variable to exceed this value, the solver is artificially blocked from finding the correct answer.
- **If M is too large:** Massive numbers destroy numerical stability. If a massive M (e.g., 10^{15}) is added to a standard coefficient like 5.2 during a row operation, the computer's finite floating-point architecture will lose the smaller number to round-off error.

The Software Heuristic: The algorithm dynamically scales M by scanning the input data before Phase I begins. A standard practice is to set M to be roughly 100 to 1,000 times larger than the maximum absolute value of the original objective coefficients ($\max |c_j|$).

2 Sparse Linear Algebra and Factorization

With the theoretical foundation for initial dual feasibility established, we must design the physical data structures that will execute the operations. We maintain a highly sparse factorization of the basis matrix $B = LU$. Understanding the underlying numerical mechanics is essential for high-performance solver architecture.

2.1 Compressed Sparse Column (CSC) Format

In large-scale optimization, the basis matrix is extremely sparse—the vast majority of its entries are zero. Storing these zeros explicitly wastes memory and computational time. The CSC format addresses this by storing *only* the non-zero elements in three contiguous arrays:

Definition — CSC Representation

A sparse matrix is stored in CSC format using three zero-indexed arrays:

1. `elements` **(Values):** A `double` array of length nnz containing the non-zero values, read column by column, top to bottom.
2. `row_indices`**:** An `int` array of length nnz giving the row index of each value in `elements`.
3. `col_starts` **(Column Pointers):** An `int` array of length $n+1$. `col_starts[j]` is the start index in `elements` for column j; `col_starts[n]` $= nnz$.

Example 2.1 — Building the CSC Arrays

Consider the following 4×4 sparse matrix A with $nnz = 6$:

$$A = \begin{bmatrix} 10.0 & 0 & 0 & 0 \\ 0 & 20.0 & 0 & 40.0 \\ 0 & 0 & 30.0 & 0 \\ 50.0 & 0 & 0 & 60.0 \end{bmatrix}$$

Extracting non-zeros **column by column** (left to right, top to bottom):

- **Column 0:** 10.0 (Row 0), 50.0 (Row 3)
- **Column 1:** 20.0 (Row 1)
- **Column 2:** 30.0 (Row 2)
- **Column 3:** 40.0 (Row 1), 60.0 (Row 3)

This translates to the three zero-indexed arrays:

$$\texttt{elements} = [10.0,\ 50.0,\ 20.0,\ 30.0,\ 40.0,\ 60.0]$$

$$\texttt{row_indices} = [0,\ 3,\ 1,\ 2,\ 1,\ 3]$$

$$\texttt{col_starts} = [0,\ 2,\ 3,\ 4,\ 6]$$

How to read `col_starts`*:* To locate Column 3, check `col_starts[3]`$=$ 4 and `col_starts[4]`$=$ 6. Column 3's data occupies indices 4 and 5 in `elements` and `row_indices`. The final entry 6 equals nnz, as required.

2.2 The Foundation: LU Factorization

Definition — LU Decomposition

LU factorization decomposes a square matrix A into the product

$$A = LU,$$

where L is *lower triangular* (ones on the diagonal, elimination multipliers below) and U is *upper triangular* (non-zeros on and above the diagonal, result of Gaussian elimination).

This factorization is highly efficient for linear systems $Ax = b$. Substituting LU for A yields $LUx = b$, solved in two fast sequential steps:

$$\boxed{Ax = b} \xrightarrow{A = LU} \boxed{\begin{array}{c} Ly = b \\ \texttt{(forward subst.)} \end{array}} \xrightarrow{\text{solve}} \boxed{\begin{array}{c} Ux = y \\ \texttt{(backward subst.)} \end{array}}$$

Example 2.2 — LU Factorization of a 3×3 Matrix

Perform LU factorization on

$$A = \begin{pmatrix} 2 & 1 & 1 \\ 4 & -6 & 0 \\ -2 & 7 & 2 \end{pmatrix}.$$

Step 1 — Eliminate below the first pivot ($a_{11} = 2$)

- $\ell_{21} = 4/2 = 2.$ $\quad \text{Row}_2 \leftarrow \text{Row}_2 - 2\,\text{Row}_1 \;\Rightarrow\; (0,\ -8,\ -2)$
- $\ell_{31} = -2/2 = -1.$ $\quad \text{Row}_3 \leftarrow \text{Row}_3 - (-1)\,\text{Row}_1 \;\Rightarrow\; (0,\ 8,\ 3)$

Step 2 — Eliminate below the second pivot ($u_{22} = -8$)

- $\ell_{32} = 8/(-8) = -1.$ $\quad \text{Row}_3 \leftarrow \text{Row}_3 - (-1)\,\text{Row}_2 \;\Rightarrow\; (0,\ 0,\ 1)$

Step 3 — Assemble L and U

$$L = \begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix}, \qquad U = \begin{pmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{pmatrix}.$$

2.3 Forward and Backward Substitution

Example 2.3 — Solving $Ax = b$ via LU

Using the factorization from Example 2.2 with $b = (4,\ -2,\ 7)^T$, we solve $LUx = b$ in two phases.

Phase 1 — Forward Substitution ($Ly = b$)

$$\begin{pmatrix} 1 & 0 & 0 \\ 2 & 1 & 0 \\ -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} = \begin{pmatrix} 4 \\ -2 \\ 7 \end{pmatrix}$$

Solving top-down: $y_1 = 4$, $y_2 = -2 - 2(4) = -10$, $y_3 = 7 + (-4) - (-10) \cdot (-1) = 1$.

$$y = (4,\ -10,\ 1)^T.$$

Phase 2 — Backward Substitution ($Ux = y$)

$$\begin{pmatrix} 2 & 1 & 1 \\ 0 & -8 & -2 \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 4 \\ -10 \\ 1 \end{pmatrix}$$

Solving bottom-up: $x_3 = 1$, $x_2 = (-10 + 2)/(-8) = 1$, $x_1 = (4 - 1 - 1)/2 = 1$.

$$\boxed{x = (1,\ 1,\ 1)^T.}$$

2.4 The Sparse Matrix Dilemma and Fill-In

In optimization, basis matrices are often large but extremely sparse. During Gaussian elimination, a zero entry can become non-zero—this is called **fill-in**.

When using a pivot element a_{pq} to update a_{ij}, the update rule is:

$$a_{ij}^{(\text{new})} = a_{ij}^{(\text{old})} - \left(\frac{a_{iq}^{(\text{old})}}{a_{pq}^{(\text{old})}} \right) a_{pj}^{(\text{old})}. \tag{1}$$

Important

Fill-in occurs when $a_{ij}^{(\text{old})} = 0$ but the multiplier a_{iq}/a_{pq} and the pivot-row element a_{pj} are *both* non-zero. Unchecked fill-in can destroy sparsity and make the factorization computationally intractable for large matrices.

2.5 Markowitz Threshold Pivoting

Markowitz Threshold Pivoting balances two competing goals: *sparsity* (minimizing fill-in) and *numerical stability* (avoiding division by very small numbers).

1. The Markowitz Count (Sparsity)

Definition — Markowitz Count

For a candidate pivot a_{ij}, let r_i be the number of non-zeros in row i and c_j the number in column j. The *Markowitz Count* is:

$$M_{ij} = (r_i - 1)(c_j - 1).$$

This gives an upper bound on the fill-in produced by choosing a_{ij} as the pivot.

2. The Threshold (Stability)

To ensure numerical stability, a candidate pivot a_{ij} must satisfy:

$$|a_{ij}| \geq u \max_k |a_{kj}|,$$

where $0 < u \leq 1$ is the threshold parameter. The algorithm selects the element with the smallest Markowitz Count M_{ij} among all elements that pass this threshold check.

Note

A typical value for the threshold parameter is $u = 0.1$. Setting $u = 1$ recovers full partial pivoting (maximally stable but potentially high fill-in), while $u \to 0$ prioritizes sparsity at the cost of stability.

2.6 Delegating Heavy Lifting to Third-Party Libraries

Established packages like **Eigen**, **SuperLU**, and **SuiteSparse (UMFPACK)** offer robust, production-grade factorization routines that perform column preordering to minimize fill-in and apply partial pivoting to maintain numerical stability. We rely on these libraries to efficiently compute the sparse LU factorization of our basis matrix B.

Note — The Key Insight of the Revised Simplex Method

We do *not* need the explicitly inverted basis B^{-1} or the dense tableau $B^{-1}A$. In any given iteration, we only need **one specific row** (the pivot row) and **one specific column** (the pivot column)—both obtained "on the fly" by solving sparse linear systems against B.

3 The Revised Dual Simplex Core Operations

Having delegated the heavy factorization to third-party libraries, we now construct the execution loop. We assume the model is in standard *minimization* form and the initial basis is *dual feasible*. The Dual Simplex algorithm maintains dual feasibility (all reduced costs $\bar{c}_j \geq 0$) while pushing toward *primal feasibility* (all $x_B \geq 0$).

3.1 Computing Pivot Row and Column

`1. CHUZR --- Choose Row (Pivot Row Selection)`

Examine the current basic variable values $x_B = B^{-1}b$.

- Identify a basic variable $x_{B(p)} < 0$ (primal infeasibility).
- This variable **leaves the basis**; index p is the *leaving row*.

2. BTRAN --- Backward Transformation

To perform the dual ratio test, we need the p-th row of the tableau $B^{-1}A$. This row equals $e_p^T B^{-1} A$. Define $\pi^T = e_p^T B^{-1}$; transposing gives:

$$B^T \pi = e_p.$$

We solve this sparse system for π. The j-th element of the pivot row is then:

$$\alpha_{pj} = \pi^T A_j.$$

3. CHUZC --- Choose Column (Dual Ratio Test)

Armed with α_{pj} and reduced costs $\bar{c}_j$:

- Consider only non-basic columns $j \in N$ where $\alpha_{pj} < 0$.
- **Exit condition:** If all $\alpha_{pj} \geq 0$, the primal problem is *infeasible* (dual unbounded). Terminate immediately.
- Choose the entering column q minimizing $\bar{c}_j / |\alpha_{pj}|$. This preserves dual feasibility after the pivot.

4. FTRAN --- Forward Transformation

Compute the entering column in tableau space, $d = B^{-1} A_q$, by solving:

$$B\, d = A_q.$$

The vector d is used to update x_B and to feed the basis refactorization (or eta-file update in Phase II).

4 Handling Hypersparsity in Triangular Solves

The standard FTRAN and BTRAN operations solve sparse systems against B. However, in massive models, we encounter an even more extreme phenomenon: **hypersparsity**.

When we solve $Bd = A_q$ (`FTRAN`), the right-hand side is a single column of the constraint matrix. In a model with millions of variables, this column might contain only two or three non-zero entries.

Definition — Hypersparsity

A vector is *hypersparse* when its number of non-zero elements (nnz) is a vanishingly small fraction of its total dimension n. In hypersparse regimes, $O(n)$ operations become computational bottlenecks, and algorithms must be designed to scale strictly with $O(nnz)$.

Important — The General-Purpose Library Trap

Standard sparse linear algebra packages (like Eigen, SuperLU, or UMFPACK) are highly optimized for factorizing sparse matrices, but their default `.solve()` routines almost always assume a *dense* right-hand side vector. If you pass a hypersparse column into a standard library's solver, it will allocate a dense accumulator array and loop from 1 to n. For a dimension of 1,000,000, this $O(n)$ overhead will instantly cripple your solver's performance.

To achieve production-grade speed, modern solvers implement customized hypersparse triangular solves using a specialized data structure.

4.1 The `IndexedVector` Data Structure

We replace standard vectors with an `IndexedVector` (often called a work vector or Sparse Accumulator). It combines the $O(1)$ lookup speed of a dense array with the $O(nnz)$ iteration speed of a packed array.

```cpp
struct IndexedVector {
    // Array of size n: Holds the actual numerical values.
    // Most elements remain exactly 0.0 at all times.
    std::vector<double> elements;

    // Array of size n: Stores the exact indices where non-zeros live.
    std::vector<int> indices;

    // Tracks how many active non-zeros currently exist.
    int nnz = 0;

    // O(nnz) reset: Clears only the active elements, avoiding O(n) memset
    void clear() {
        for (int i = 0; i < nnz; ++i) {
            elements[indices[i]] = 0.0;
        }
        nnz = 0;
    }
};
```

Listing 1: C++ Blueprint for an IndexedVector

4.2 The Gilbert-Peierls Algorithm

With the `IndexedVector` tracking exactly where our non-zeros are, we can execute the hypersparse triangular solve. This relies on the **Gilbert-Peierls Algorithm**, which treats the lower-triangular matrix L as a Directed Acyclic Graph (DAG) and solves the system in two distinct phases.

Phase 1: Reachability (Topological Sort)

Before performing any floating-point arithmetic, the solver must determine exactly which variables will become non-zero during the substitution.

- Execute a Depth-First Search (DFS) on the graph of L, starting *only* from the non-zero indices provided by the initial right-hand side vector.
- As the DFS backtracks, record the visited nodes. This generates a topologically sorted list of indices.

- This list contains only the variables that will be active, sorted into the exact mathematical order required by forward substitution.

Phase 2: Numerical Solve

The solver now computes the values, iterating strictly over the reachable indices identified in Phase 1.

- Loop through the topologically sorted list. For each index i, read `elements[i]`.
- Multiply it against the non-zero entries in column i of L, and subtract the results from the corresponding indices in the `elements` array.
- If this subtraction writes a value to a previously zero element, it is recorded as "fill-in" and added to the `indices` array.

4.3 Viewing the Matrix as a Graph

To understand how we can magically skip the $O(n)$ iteration during a sparse solve, we must stop looking at the lower-triangular matrix L as a grid of numbers, and start viewing it as a **dependency network**.

Definition — Directed Acyclic Graph of L

Every row/column index in the matrix represents a "node" (a variable x_i). A non-zero element ℓ_{ij} in the matrix means that the equation for row i contains the variable x_j.
Therefore, calculating x_i *depends* on first knowing x_j. We represent this as a directed edge from node j to node i ($j \to i$). Because L is strictly lower-triangular, all edges flow from smaller indices to larger indices. It is impossible to create a loop, making this a Directed Acyclic Graph (DAG).

4.4 Depth-First Search (DFS) and Topological Sort

During `FTRAN` ($Ld = A_q$), we must perform forward substitution. In a dependency network, this means we must solve variables in a **topological order**—a strict sequence where every prerequisite variable is solved before any variable that depends on it.

To find this order efficiently, we use a classic computer science algorithm called **Depth-First Search (DFS)**.

The DFS Post-Order Algorithm

DFS explores a graph by starting at a given node and traveling as far down a path as possible before retreating. To generate our topological order:

1. **Start:** Begin at the non-zero indices provided by the initial right-hand side vector A_q.
2. **Dive:** Follow outgoing edges recursively to unvisited children.
3. **Backtrack & Record:** When a node has no unvisited children (it is a dead end), we backtrack. *Only at the moment of backtracking do we append the node*

to our list.

4. **Reverse:** Because a node is recorded only after all its dependencies (children) have been fully explored, the final list is in *reverse* topological order. Reversing this list yields the perfect forward substitution sequence.

4.5 Step-by-Step Reachability Example

Example 4.1 — DFS Reachability on a 5×5 Matrix

Consider a lower-triangular matrix L and a hypersparse right-hand side vector b:

$$L = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 \\ 2 & 0 & 1 & 0 & 0 \\ 0 & 4 & 0 & 1 & 0 \\ 0 & 0 & 5 & 0 & 1 \end{pmatrix}, \qquad b = \begin{pmatrix} 7 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Notice the dependencies (edges) defined by the non-zeros below the diagonal:

- $\ell_{31} = 2 \implies$ Edge $1 \to 3$
- $\ell_{42} = 4 \implies$ Edge $2 \to 4$
- $\ell_{53} = 5 \implies$ Edge $3 \to 5$

Execution of DFS: We start *only* at the non-zero indices of b. Since only $b_1 \neq 0$, our DFS starts exclusively at Node 1.

1. **Visit 1:** Check outgoing edges. Node 1 connects to Node 3.
2. **Visit 3:** Check outgoing edges. Node 3 connects to Node 5.
3. **Visit 5:** Check outgoing edges. None exist. It is a dead end.
4. **Backtrack 5:** Record Node 5. (List: `[5]`)
5. **Backtrack 3:** Record Node 3. (List: `[5, 3]`)
6. **Backtrack 1:** Record Node 1. (List: `[5, 3, 1]`)

Reversing the list yields the final topological solve order: `[1, 3, 5]`.

Note — Why We Only Solve Reachable Indices

In Example 4.5, the DFS never touched Node 2 or Node 4. Why is it mathematically safe to ignore them during forward substitution?
Look at the equation for variable x_2: $1 \cdot x_2 = b_2$. Because $b_2 = 0$, x_2 must be 0.
Look at the equation for x_4: $4 \cdot x_2 + 1 \cdot x_4 = b_4$. Because $b_4 = 0$ and $x_2 = 0$, x_4 must also be exactly 0.
If the initial right-hand side value is zero, and no upstream variable that feeds into it is non-zero, the variable will provably remain zero.

5 Phase II: Advanced Basis Maintenance (ETA-Files)

Hypersparse solves drastically accelerate individual iterations. However, relying on a third-party library to compute the sparse factorization $B = LU$ from scratch *every iteration* remains highly inefficient. During a Simplex iteration, the basis matrix B changes by exactly one column (the leaving basic variable is replaced by the entering variable). Modern solvers maintain the basis updates efficiently using the **Product Form of the Inverse (PFI)** and **ETA-files**.

5.1 The Mathematical Derivation of the Update

Let B be our current basis matrix, and let p be the index of the leaving row. This means we are removing the p-th column of B and replacing it with the entering column A_q to form our new basis, B_{new}. We can express this column replacement mathematically:

$$B_{\text{new}} = B + (A_q - Be_p)e_p^T$$

Factoring out B on the left gives:

$$B_{\text{new}} = B\left(I + (B^{-1}A_q - e_p)e_p^T\right)$$

Recall our `FTRAN` step. The term $B^{-1}A_q$ is exactly the vector d that we already computed! Substituting d into the equation yields:

$$B_{\text{new}} = B\underbrace{\left(I + (d - e_p)e_p^T\right)}_{E}$$

Definition — The Eta Matrix (E)

The matrix $E = I + (d - e_p)e_p^T$ is called an *Eta Matrix*. It is identical to the Identity matrix in every column except for column p, which contains the `FTRAN` vector d. The vector d is referred to as the *eta vector* (η).

5.2 The ETA-File Structure

The brilliance of this derivation is that to update our basis, we do not need to alter L or U, nor do we need to store a dense B_{new}^{-1}. We only need to store the vector d and its pivot index p.

As the solver progresses through iterations $k = 1, 2, \ldots, K$, it simply appends each new eta vector to a sequential list known as the **ETA-file**. The true inverse of the current basis is represented implicitly as a chain of matrix multiplications:

$$B_K^{-1} = E_K^{-1}E_{K-1}^{-1}\ldots E_1^{-1}U^{-1}L^{-1}$$

`FTRAN` and `BTRAN` with ETA-Files

With the ETA-file in place, our triangular solves require an extra loop to process the accumulated eta matrices.

`FTRAN` **($B_K d = A_q$):**

1. Solve $Ly = A_q$
2. Solve $Uv = y$
3. For $k = 1$ to K: Solve $E_k x = v$ $\quad\leftarrow$ (Forward sweep)

`BTRAN` ($B_K^T \pi = e_p$):

1. For $k = K$ down to 1: Solve $E_k^T y = e_p$ $\quad \leftarrow$ (Backward sweep)
2. Solve $U^T v = y$
3. Solve $L^T \pi = v$

5.3 Solving against an Eta Matrix

Solving $Ex = v$ is incredibly fast because E is almost entirely the Identity matrix. We do not need a heavy linear algebra package; we execute a simple $O(nnz)$ loop over the eta vector.

Example 5.1 — Solving $Ex = v$

Let E be an eta matrix where column $p = 2$ has been replaced by the eta vector $d = (2, -4, 3)^T$. We want to solve $Ex = v$ where $v = (10, 8, 12)^T$.

$$\begin{pmatrix} 1 & 2 & 0 \\ 0 & -4 & 0 \\ 0 & 3 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 10 \\ 8 \\ 12 \end{pmatrix}$$

Because of the structure of E, we always solve for x_p first:

$$-4x_2 = 8 \implies x_2 = -2$$

Then, we substitute x_2 to find the remaining variables:

$$x_1 + 2(-2) = 10 \implies x_1 = 14$$
$$3(-2) + x_3 = 12 \implies x_3 = 18$$

In software, this is executed effortlessly using the `IndexedVector` containing d.

Important — The Refactorization Trigger

The ETA-file cannot grow indefinitely. As we append more eta vectors:

- The cost of `FTRAN` and `BTRAN` increases with every iteration, slowing down the solver.
- The eta vectors become increasingly dense (losing hypersparsity).
- Floating-point round-off errors accumulate, degrading the numerical stability of the solutions.

To fix this, production solvers monitor the length of the ETA-file and the numerical error. When a threshold is reached (e.g., 100 iterations), the solver **clears the ETA-file completely** and commands the third-party library to compute a fresh, highly sparse LU factorization of the current basis matrix B.

6 Phase III: Advanced Dual Pricing and Ratio Tests

In the standard textbook Dual Simplex method, the algorithm begins by finding a leaving variable (`CHUZR`). The classic rule is to select the most primal infeasible basic variable: choose row p such that $x_{B(p)}$ is the most negative. Once the row is chosen, the entering column (`CHUZC`) is selected using the standard dual ratio test.

While these textbook rules are easy to compute, they fail spectacularly in large-scale production solvers. They suffer from geometric scale illusions and numerical degeneracy. To achieve high performance, modern solvers replace these rules with **Dual Steepest Edge** and the **Harris Two-Pass Ratio Test**.

6.1 Dual Steepest Edge (`CHUZR`)

The classic rule of choosing the most negative $x_{B(i)}$ is flawed because it evaluates the primal infeasibility in isolation, ignoring the geometry of the dual space.

Important — The Scale Illusion in Dual Space

Just because a basic variable is deeply negative does not mean it will yield a good step. If the normal vector associated with that row is massive, the actual step taken in the dual objective space (θ_d) will be vanishingly small. The solver will execute an expensive pivot that barely improves the dual objective.

To fully grasp the mechanics of Dual Steepest Edge, we will show why the dual variables move precisely along the vector $e_p^T B^{-1}$ when the p-th basic variable leaves the basis. This requires understanding what it geometrically means to "leave a basis" in the dual space.

1. The Starting Vertex (Tight Constraints)

In the dual space, the variables are represented by the vector π. The dual constraints are defined by the columns of the primal matrix: $\pi^T A_j \leq c_j$.

At any basic feasible solution, the current dual variables (let us call them π_0) are uniquely determined by the m basic variables. Geometrically, π_0 sits at the exact intersection of m tight hyperplanes. Mathematically, this means π_0 perfectly satisfies the equality equations for all basic columns:

$$\pi_0^T B_i = c_{B(i)} \quad \text{for all } i = 1, \ldots, m$$

2. Leaving the Basis (Moving Along an Edge)

In the Dual Simplex method, selecting row p to leave the basis means we are actively dropping the p-th basic variable.

Geometrically, we are intentionally relaxing the p-th tight constraint, allowing the dual variables to move away from it. However, to stay on an *edge* of the dual polyhedron, the new dual variables must remain strictly glued to the remaining $m-1$ hyperplanes.

Let the new dual variables move along a ray parameterized by a step size $\theta > 0$ and a direction vector η:

$$\pi(\theta) = \pi_0 + \theta\eta$$

Because we must stay on the remaining $m-1$ hyperplanes, this new ray must still perfectly satisfy their equations:

$$(\pi_0 + \theta\eta)^T B_i = c_{B(i)} \quad \text{for all } i \neq p$$

3. The Orthogonality Requirement

Let us expand the constraint equation for the ray:

$$\pi_0^T B_i + \theta \eta^T B_i = c_{B(i)}$$

Because we established in Step 1 that $\pi_0^T B_i = c_{B(i)}$, we can subtract it from both sides:

$$\theta \eta^T B_i = 0$$

Since we are taking a strictly positive step ($\theta > 0$), the math demands a strict condition on our direction vector η:

$$\eta^T B_i = 0 \quad \text{for all } i \neq p$$

Definition — The Orthogonality Condition

To travel along a dual edge by dropping the p-th basic variable, the direction vector η^T must be strictly orthogonal to every single column of the basis matrix B, *except* for the p-th column.

4. The Magic of the Inverse Matrix

How do we find a vector that is perfectly orthogonal to $m-1$ specific columns of a matrix? We rely on the fundamental definition of the matrix inverse: $B^{-1}B = I$.

Let ρ_k be the k-th row of B^{-1}. If we multiply this row against the i-th column of B, the result must exactly match the identity matrix:

$$\rho_k B_i = 1 \quad \text{if } k = i$$
$$\rho_k B_i = 0 \quad \text{if } k \neq i$$

We can algebraically extract the p-th row of the inverse matrix by multiplying B^{-1} by the transposed standard unit vector e_p^T:

$$\rho_p = e_p^T B^{-1}$$

According to the identity matrix rule, this specific row vector satisfies exactly the condition we derived in Step 3:

$$(e_p^T B^{-1}) B_i = 0 \quad \text{for all } i \neq p$$

Note — Conclusion of the Proof

Because the vector $e_p^T B^{-1}$ is uniquely orthogonal to all the remaining $m-1$ basic columns, any movement that relaxes the p-th constraint while keeping the others tight **must** be a scalar multiple of this vector. Therefore, the edge direction η^T is strictly proportional to the p-th row of the inverse basis matrix.

Definition — Dual Steepest Edge Criterion

The Dual Steepest Edge (DSE) rule chooses the leaving row p that maximizes the rate of change of the dual objective. Mathematically, it selects the row that maximizes the ratio of the squared primal infeasibility to the squared Euclidean

length (L_2 norm) of the dual edge direction:

$$p = \text{argmax}_{i \in \mathcal{B},\, x_{B(i)} < 0} \frac{x_{B(i)}^2}{\|\rho_i\|_2^2}$$

Implementations define a *Dual Steepest Edge Weight* $w_i = \|e_i^T B^{-1}\|_2^2$. Notice that $e_i^T B^{-1}$ is exactly the vector computed during the `BTRAN` step!

6.2 The Goldfarb-Reid Recurrence Relation

The Goldfarb-Reid recurrence relation (introduced by Donald Goldfarb and John Reid in 1977) is the mathematical engine that makes Steepest Edge pricing computationally viable. It allows the solver to update the exact geometric weights of the dual space using the vectors already computed during the pivot.

Our goal is to update the Dual Steepest Edge weights, $w_i = \|\rho_i\|_2^2$, where $\rho_i = e_i^T B^{-1}$ is the i-th row of the inverse basis matrix.

1. The Algebraic Update of the Inverse Rows

When we pivot on row p and column q, the basis inverse is updated using the eta matrix E. Recall our `FTRAN` vector is $d = B^{-1} A_q$, and the pivot element is d_p. The mathematical update for the rows of the new inverse basis matrix ($\hat{B}^{-1}$) is a standard Gaussian elimination step:

$$\hat{\rho}_p = \frac{1}{d_p} \rho_p \qquad \text{(The new pivot row)}$$

$$\hat{\rho}_i = \rho_i - \frac{d_i}{d_p} \rho_p \qquad \text{(All other rows, } i \neq p\text{)}$$

2. Expanding the Squared Norm

To find the new weight $\hat{w}_i = \|\hat{\rho}_i\|_2^2$, we take the dot product of the updated row with itself:

$$\hat{w}_i = \left(\rho_i - \frac{d_i}{d_p}\rho_p\right)\left(\rho_i - \frac{d_i}{d_p}\rho_p\right)^T$$

Expanding this binomial yields three terms:

$$\hat{w}_i = (\rho_i \rho_i^T) - 2\frac{d_i}{d_p}(\rho_i \rho_p^T) + \left(\frac{d_i}{d_p}\right)^2 (\rho_p \rho_p^T)$$

Notice the brilliance of this expansion. The first term $(\rho_i \rho_i^T)$ is exactly our old weight w_i. The last term $(\rho_p \rho_p^T)$ is exactly our old pivot row weight w_p.

Substituting these known values gives us the skeleton of the recurrence relation:

$$\hat{w}_i = w_i - 2\frac{d_i}{d_p}(\rho_i \rho_p^T) + \left(\frac{d_i}{d_p}\right)^2 w_p$$

3. The Missing Cross-Term and the Extra Solve

The only unknown in our formula is the cross-term $(\rho_i \rho_p^T)$, which is the dot product of the i-th row and the p-th row of the inverse basis. Let us define a new vector v to hold all these cross-terms:

$$v_i = \rho_i \rho_p^T$$

Substituting the definition of $\rho_i = e_i^T B^{-1}$, we get:

$$v_i = (e_i^T B^{-1}) \rho_p^T$$

Because e_i^T simply extracts the i-th element of a vector, the entire vector v can be written as:

$$v = B^{-1} \rho_p^T$$

Note — The Computational Tradeoff

Look closely at the equation $v = B^{-1}\rho_p^T$. This is mathematically equivalent to solving the linear system $Bv = \rho_p^T$.
This means v can be computed by executing a **single `FTRAN` operation**, using the transposed `BTRAN` vector (ρ_p^T) as the right-hand side! This is the fundamental tradeoff of Steepest Edge: we trade the impossible task of executing m `BTRAN` solves for the cost of executing just *one* extra `FTRAN` solve per iteration.

The Goldfarb-Reid Weight Update

At the end of a Dual Simplex iteration, before advancing to the next, update the weight array as follows:

1. **The Extra Solve:** Execute `FTRAN` using the BTRAN vector to find $v = B^{-1}\rho_p^T$.
2. **Update the Pivot Row:**
$$\hat{w}_p = \frac{1}{d_p^2} w_p$$
3. **Update the Remaining Rows:** Loop over the non-zero elements of the FTRAN vector d (using our `IndexedVector`). For each $i \neq p$ where $d_i \neq 0$:
$$\hat{w}_i = \max\left(1.0,\ w_i - 2\frac{d_i}{d_p} v_i + \left(\frac{d_i}{d_p}\right)^2 w_p\right)$$

Note: The $\max(1.0, \dots)$ *guard is used in production software because floating-point round-off errors can occasionally cause a weight to drop below its theoretical minimum of* 1.0 *(*$w_i \geq 1$ *because the identity matrix is embedded within the basis update).*

Note — Numerical Implementation: Dynamic Weight Maintenance

To maintain the efficiency of the Dual Steepest Edge (`DSE`) rule, weights $w_i = \|e_i^T B^{-1}\|_2^2$ are updated dynamically rather than recomputed from scratch:

- **The Refactorization Reset:** Exact weights are computed using m backward substitutions only when the ETA-file is cleared and the basis is refactored (see Phase II). This establishes a numerically precise baseline.
- **Recurrence Updates:** Between refactorizations, the solver applies the

Goldfarb-Reid recurrence relation. This updates the weight array w using the pivot row α_p and the `FTRAN` vector d, both of which are already computed during the current iteration.

- **Hypersparse Update:** The update is executed using the `IndexedVector` (or Sparse Accumulator) architecture, ensuring that the weight maintenance step scales with the number of non-zeros (nnz) rather than the full dimension n.

By tying weight management to the basis maintenance cycle, `DSE` provides a superior geometric path with almost zero additional computational overhead per iteration.

6.3 The Harris Two-Pass Ratio Test (`CHUZC`)

Once the leaving row p is chosen via Dual Steepest Edge, we compute the pivot row $\alpha_p = \rho_p A$. The classic dual ratio test then finds the entering column q that minimizes $\bar{c}_j/|\alpha_{pj}|$ to preserve dual feasibility.

However, in massive models, you will encounter *degeneracy*: many columns will tie for the minimum ratio. If the standard rule picks a column with a very small pivot element $|\alpha_{pj}|$, dividing by this tiny number during the `FTRAN` update will cause catastrophic floating-point explosion.

In 1973, Paula Harris solved this by introducing a robust, two-pass ratio test that leverages a tiny dual feasibility tolerance (ϵ_d, typically 10^{-7}).

The Harris Two-Pass Algorithm

Pass 1: Determine the Maximum Step Size (Δ) Instead of finding the absolute minimum ratio, we allow the reduced costs to become ever-so-slightly negative (up to $-\epsilon_d$). We compute the maximum acceptable dual step size:

$$\Delta = \min_{j \in \mathcal{N},\, \alpha_{pj} < 0} \frac{\bar{c}_j + \epsilon_d}{|\alpha_{pj}|}$$

Any column j whose ratio $\bar{c}_j/|\alpha_{pj}|$ is less than or equal to Δ is mathematically capable of maintaining dual feasibility within our strict tolerance.

Pass 2: Maximize Numerical Stability We now have a pool of candidate columns that satisfy the Δ threshold. From this pool, we select the entering column q that has the *largest absolute pivot element*:

$$q = \text{argmax}_{j \in \text{candidates}} |\alpha_{pj}|$$

By deliberately allowing a microscopic, mathematically controlled violation of dual feasibility, the Harris Two-Pass test completely breaks degenerate ties and guarantees that the solver always divides by the most stable, largest available pivot element.

7 Phase 0: Presolve and Postsolve Architecture

In real-world linear programming models, constraints are often generated by automated modeling languages. This routinely results in massive matrices filled with redundant equations, fixed variables, and structurally empty rows.

Feeding this raw data directly into the Simplex algorithm wastes immense computational time. Modern solvers bypass this by executing a **Presolve** phase—a recursive loop of logical deductions that shrinks the matrix before optimization begins. Once the simplified problem is solved, a **Postsolve** routine maps the solution back to the original variables.

7.1 Standard Form Presolve Reductions

Because our variables are strictly bounded by $x_j \geq 0$, the logical deductions are purely algebraic. The solver loops through the following checks until no further reductions can be made.

1. Empty Rows and Columns

The simplest reduction removes constraints or variables that contain no data.

- **Empty Row:** A row i where $a_{ij} = 0$ for all j. The constraint is mathematically $0 = b_i$. If $b_i \neq 0$, the solver instantly terminates and declares the problem **Primal Infeasible**. If $b_i = 0$, the constraint is redundant and row i is deleted.
- **Empty Column:** A column j where $a_{ij} = 0$ for all i. The variable x_j does not appear in any constraint, meaning its value is governed entirely by its objective coefficient c_j. Because $x_j \geq 0$, if $c_j < 0$, we can increase x_j to infinity for infinite profit. The solver terminates and declares the problem **Unbounded**. If $c_j \geq 0$, the optimal choice is to not use the variable. We fix $x_j = 0$ and delete column j.

2. Singleton Rows (Fixed Variables)

A singleton row is a constraint containing exactly one non-zero coefficient. It forces a variable to take a specific mathematical value.

Definition — Row Singleton Elimination

Let row i contain exactly one non-zero entry, a_{ik}. The constraint simplifies to $a_{ik}x_k = b_i$, meaning the variable is fixed at:

$$x_k = \frac{b_i}{a_{ik}}$$

Because we are in standard form, if $x_k < 0$, the solver terminates and declares the problem **Primal Infeasible**. If $x_k \geq 0$, the solver executes a permanent substitution:

1. Subtract $(a_{mk} \cdot x_k)$ from the right-hand side b_m for all other rows $m \neq i$.
2. Subtract $(c_k \cdot x_k)$ from the objective offset.
3. Delete row i and column k entirely from the matrix.

3. Forcing Rows

Because every variable must be non-negative ($x \geq 0$), the signs of the coefficients can trap variables at zero.

The Forcing Zero Rule

Consider a constraint i where the right-hand side is zero ($b_i = 0$). If all non-zero coefficients in that row share the same sign (e.g., all $a_{ij} > 0$), the only way the sum can equal zero without negative variables is if **every single variable in that row is strictly zero**.
For every j where $a_{ij} \neq 0$:

- Fix $x_j = 0$.
- Delete column j from the matrix.

Finally, delete the now-empty row i.

7.2 The Postsolve Stack

Every time the Presolve loop deletes a row or a column, it must record the exact algebraic relationship it destroyed. It pushes these operations onto a Last-In-First-Out (LIFO) data structure known as the **Postsolve Stack**.

When the Simplex algorithm finishes solving the reduced "core" problem, the solver must reconstruct the full solution. The primal variables (x) are easy to recover via back-substitution. However, recovering the optimal *dual variables* (π) is the true engineering challenge of the Postsolve phase.

Example 7.1 — Recovering Dual Variables from a Singleton

Suppose Presolve found a singleton row i ($2x_k = 10$), fixed $x_k = 5$, and deleted row i and column k.
After the core problem is optimized, we have the dual variables π for the surviving rows. We must calculate the missing dual variable π_i for the row we deleted.
By the definition of dual feasibility, the reduced cost of the deleted column k must satisfy $\bar{c}_k \geq 0$. Since $x_k = 5$ (it is strictly positive), Complementary Slackness dictates that its reduced cost must be exactly zero!

$$\bar{c}_k = c_k - \sum_{m \neq i} \pi_m a_{mk} - \pi_i a_{ik} = 0$$

Because π_i is the only unknown in this equation, the Postsolve routine algebraically isolates it:

$$\pi_i = \frac{1}{a_{ik}} \left(c_k - \sum_{m \neq i} \pi_m a_{mk} \right)$$

Note — The LIFO Execution

Because variables deleted early in Presolve often depend on variables deleted later, the Postsolve routine must pop operations off the stack in reverse order (LIFO). The solver carefully steps backward through time, perfectly reconstructing the primal and dual vectors until the original dimensions of the problem are fully restored.

8 System Architecture and Modular Design

8.1 Module 1: The Data Model (`LpData`)

Inspired by `HighsLp` and `HighsSparseMatrix`, this module manages the static problem data and memory pools. It strictly separates the immutable problem definition from the active solver state.

- **`CscMatrix` Struct:** Implements the Compressed Sparse Column format using three tightly packed `std::vector` arrays: `elements` (`double`), `row_indices` (`int`), and `col_starts` (`int`).
- **`LpModel` Class:** The immutable container for the problem. It holds the constraint matrix A, the right-hand side vector b, objective coefficients c, and the integer dimensions m and n.
- **`IndexedVector` Pool:** The primary sparse accumulator for $O(nnz)$ operations.

> **Note — Memory Strategy**
>
> These vectors are allocated exactly once per thread/solve at startup to entirely avoid dynamic heap allocations (`new` or `std::vector::resize`) during the fast Simplex loop.

8.2 Module 2: The Factorization Engine (`LpFactor`)

Inspired by `HFactor`, this module is a completely isolated linear algebra engine. The Simplex solver never touches the basis matrix directly; it communicates with this module purely via abstract `ftran()` and `btran()` requests.

- **Third-Party LU Interface:** Accepts an array of basic column indices, constructs the basis B, and delegates to an external library (e.g., SuiteSparse/UMFPACK) for Markowitz threshold pivoting and factorization. It extracts the resulting L and U factors back into internal `CscMatrix` formats.
- **ETA-File Manager (Product Form of the Inverse):** Stores an expanding `std::vector<EtaMatrix>`. It tracks the density of the ETA-file and numerical round-off errors. When the threshold is reached, it clears the file and commands the LU Interface to compute a fresh factorization.
- **Hypersparse Solvers:** Implements the Gilbert-Peierls algorithm. Uses a private Depth-First Search (DFS) routine to compute the topological sort on the L and U DAGs, executing numerical substitutions strictly over the reachable indices.

8.3 Module 3: The Dual Simplex Engine (`DualSimplexEngine`)

Inspired by `HEkk` (the HiGHS dual edge solver), this is the algorithmic controller. It holds the active state of the optimization process and executes the Phase I and Phase II logic.

- **`SimplexState` Struct:** Maintains the mutable solver state: the primal values of basic variables (x_B), dual variables (π), reduced costs ($\bar{c}$), and arrays tracking the basis status of each variable.
- **`PricingManager`:**

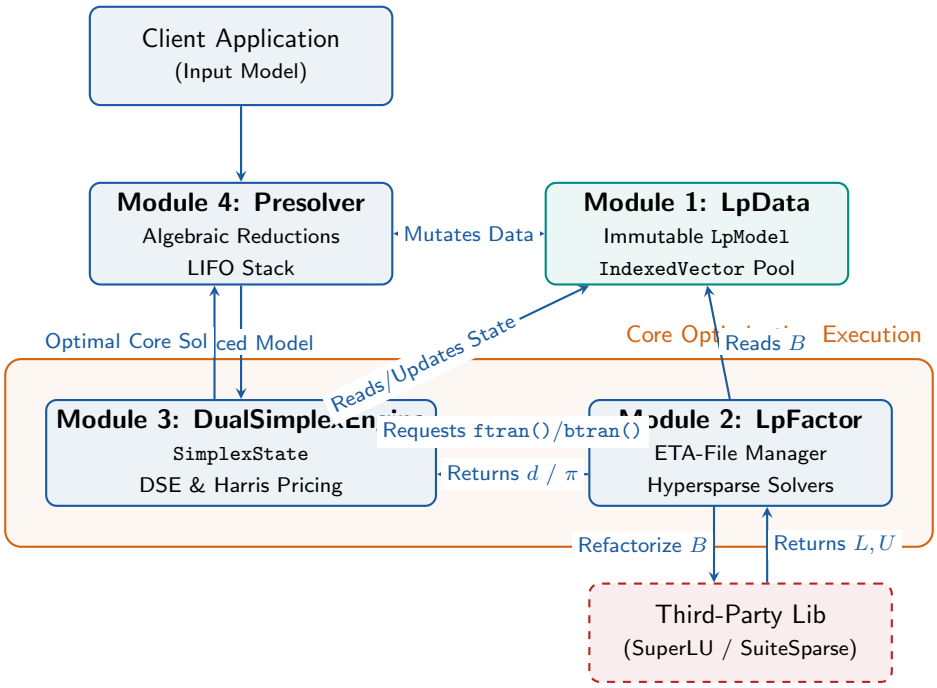
- *CHUZR (Dual Steepest Edge):* Executes the Goldfarb-Reid recurrence relation to dynamically update the geometric weights w_i in $O(nnz)$ time.
- *CHUZC (Harris Two-Pass):* Evaluates the maximum stable step size Δ using the dual feasibility tolerance ϵ_d, selecting the largest absolute pivot element from the candidate pool.

- `PhaseOneManager` **(Big-M):** Detects initial dual infeasibility. If found, it dynamically calculates a safe Big-M penalty (scaled to $\max|c_j|$), injects the bounding constraint into the active state, and executes the forcing pivot.
- **The Iteration Loop:** Orchestrates the strict sequence: `CHUZR` $\rightarrow$ `BTRAN` $\rightarrow$ `CHUZC` $\rightarrow$ `FTRAN` $\rightarrow$ `Update`.

8.4 Module 4: Presolve and Postsolve (`Presolver`)

Inspired by `HighsPresolve`, this module acts as a protective wrapper around the core solver, ensuring the Simplex engine only processes structurally necessary mathematics.

- `PresolveLoop`**:** Iterates over the `LpModel` applying purely algebraic standard-form reductions: empty row/column elimination, row singleton substitution, and forcing row elimination. It physically shrinks the `CscMatrix`.
- `PostsolveStack`**:** A strictly LIFO (`std::stack`) structure that records the algebraic properties of every eliminated constraint and variable.
- `PostsolveRoutine`**:** Triggered after the core solver returns the optimal solution. Pops operations in reverse-time order to reconstruct the full primal vector x and algebraically back-substitute the missing dual variables π using complementary slackness.

8.5 Architecture Diagram

The following diagram illustrates the data flow, dependency boundaries, and execution pipeline of the solver, reflecting the strict isolation of the linear algebra engine from the algorithmic core.